AI Engineering
RAG Practice Guide

A sample knowledge document for building and testing a PDF question-answering chatbot

Purpose	Practice PDF text extraction, chunking, embeddings, vector search, and retrieval-augmented generatio
Level	Beginner to intermediate
Suggested filename	ai_engineering_rag_practice_guide.pdf
Document type	Original educational practice material

This document is intentionally organized into clear sections with definitions, examples, comparisons, and practical details. It is suitable for testing basic retrieval systems. It is not a substitute for primary research papers or production documentation.

Contents

- 1. Artificial Intelligence, Machine Learning, and Deep Learning
- 2. Large Language Models and Transformers
- 3. Tokens, Context Windows, and Prompting
- 4. Embeddings and Semantic Similarity
- 5. Retrieval-Augmented Generation (RAG)
- 6. PDF Ingestion, Chunking, and Metadata
- 7. Vector Search and FAISS
- 8. Building a Reliable RAG Application
- 9. Glossary
- 10. Questions for testing your chatbot

How to use this guide: upload the PDF to your chatbot, then ask questions that target individual sections. Try both direct factual questions and questions that require combining two related concepts.

1. Artificial Intelligence, Machine Learning, and Deep Learning

Artificial Intelligence (AI) is the broad field of building computer systems that perform tasks commonly associated with human intelligence. Examples include recognizing images, understanding language, planning routes, recommending products, and assisting with decisions.

Machine Learning (ML) is a subset of AI. Instead of writing every rule manually, developers train a model using data so that it can learn patterns and make predictions on new examples. A spam classifier, for example, can learn from messages labelled as spam or not spam.

Deep Learning is a subset of machine learning that uses neural networks with multiple layers. Deep learning is particularly useful for complex data such as images, audio, and natural language.

A simple comparison

Field	Main idea	Example
Artificial Intelligence	Systems that perform intelligent tasks	A route-planning assistant
Machine Learning	Learning patterns from data	Predicting house prices
Deep Learning	Learning with multi-layer neural networks	Recognizing objects in photographs

Training and inference

Training is the process in which a model adjusts its internal parameters using examples. Inference is the process of using a trained model to produce a prediction or response. Training can be computationally expensive; inference is the step that usually happens when an application user sends a request.

A model's performance depends on the quality and relevance of its training data, its architecture, the training procedure, and how it is evaluated. A model can perform well on familiar examples but poorly on unfamiliar or biased data.

2. Large Language Models and Transformers

A **Large Language Model (LLM)** is a neural network trained on large collections of text and other data to process and generate language. Many LLMs are trained to predict the next token in a sequence. By repeatedly predicting tokens, the model can produce paragraphs, explanations, summaries, and code.

A **token** is a unit of text processed by a language model. A token may represent a whole short word, part of a longer word, punctuation, or another text fragment. Tokens are not exactly the same as words.

Transformer architecture

The Transformer is a neural-network architecture introduced for sequence modelling. Its central mechanism is attention, which allows the model to weigh relationships between different positions in an input sequence. Transformers can process relationships across a sequence more effectively than many earlier recurrent approaches.

Self-attention lets each token build a representation using information from other tokens in the sequence. For example, in the sentence “The animal did not cross the road because it was tired,” attention mechanisms can help a model relate “it” to “the animal.”

A Transformer commonly contains attention layers, feed-forward neural-network layers, normalization, and residual connections. The exact architecture differs between model families. Decoder-only Transformers are widely used for text generation.

What an LLM can and cannot do

- LLMs can summarize, classify, transform, explain, and generate text based on learned patterns.
- LLMs may produce incorrect or fabricated statements. This is often called hallucination.
- An LLM's built-in knowledge may not include recent events or private organizational documents.
- A well-designed application should validate important outputs and provide relevant source context where possible.

3. Tokens, Context Windows, and Prompting

A **context window** is the amount of token information a model can consider in one request. The context may include system instructions, conversation history, the current user question, retrieved document passages, and space needed for the model's answer.

If an application sends too much text, it may exceed the model's context limit. Even before reaching the limit, large prompts can increase latency and cost. RAG systems therefore retrieve a small number of relevant passages rather than sending every document on every question.

Prompt components

A prompt is the input that guides a model's response. In a chat application, messages often have roles such as system, user, and assistant. A system message sets high-level behaviour; a user message contains the current request; an assistant message contains a prior model response.

A useful instruction is specific about the task, the available evidence, the desired format, and what to do when the evidence is insufficient. For a document question-answering chatbot, an instruction might say: answer using the supplied document excerpts and state when the excerpts do not contain the answer.

Prompting improves how a model uses the information it receives, but it does not guarantee correctness. Applications should test prompts with expected questions and edge cases.

4. Embeddings and Semantic Similarity

An **embedding** is a numerical representation of an item, such as a sentence, paragraph, image, or other data. A text embedding model maps text into a vector: a list of numbers in a high-dimensional space.

The purpose of an embedding is to represent useful properties of the input so that related items are often located near one another in the vector space. For example, “How can I reset my password?” and “I forgot my login credentials” may have similar embeddings even though they do not share many exact words.

Similarity measures

Cosine similarity compares the angle between two vectors. It is commonly used for text embeddings. A higher cosine similarity often indicates that two texts are more semantically related, depending on the embedding model and its training.

Euclidean distance measures straight-line distance between vectors. Some vector indexes use distance-based metrics to find nearest neighbours. The choice of metric should match the embedding model's recommendations and the index configuration.

Embeddings are model-dependent. Vectors produced by different embedding models should not be mixed in the same index as if they shared the same coordinate system. If the embedding model changes, stored document vectors generally need to be regenerated.

Example

Suppose a knowledge base contains these passages: (1) “FAISS is a library for efficient similarity search,” (2) “A banana is a tropical fruit,” and (3) “Vector indexes can retrieve nearby embeddings.” A query asking “How do I search similar vectors?” should retrieve passages 1 and 3 before passage 2.

5. Retrieval-Augmented Generation (RAG)

Retrieval-Augmented Generation (RAG) combines information retrieval with text generation. Instead of relying only on information stored in the LLM's parameters, an application searches an external knowledge collection and supplies relevant passages to the model as context.

The two main RAG phases

Ingestion: documents are loaded, text is extracted, text is split into chunks, metadata is attached, embeddings are generated, and vectors are indexed for search.

Question answering: a user question is embedded, the vector index retrieves similar chunks, optional filtering or reranking improves the results, and the selected context is passed to the LLM with the question.

Why use RAG?

- It can ground answers in a private or specialized document collection.
- The knowledge collection can be updated without retraining the LLM.
- Retrieved passages can be shown as sources to help users verify answers.
- It can reduce unsupported answers when retrieval is relevant and prompts require evidence.

RAG limitations

RAG does not guarantee factual answers. Retrieval can miss the right passage, chunks can lose important context, and a language model can misinterpret the retrieved text. Poorly scanned PDFs may produce bad extraction. A reliable system measures retrieval quality and answer quality separately.

6. PDF Ingestion, Chunking, and Metadata

PDF ingestion is the process of converting a PDF into text and useful metadata for downstream search. A text-based PDF often contains extractable characters. A scanned PDF may contain only page images and require Optical Character Recognition (OCR). Tables, columns, headers, footers, and mathematical notation can be difficult to extract cleanly.

Chunking

Chunking divides long text into smaller passages. Each passage becomes a candidate retrieval unit. Chunking is important because a full document may be too large or too broad to include in a model prompt.

Chunk size is the approximate amount of text in each passage. Small chunks can be precise but may omit surrounding explanation. Large chunks preserve more context but may retrieve irrelevant text and consume more context-window space.

Chunk overlap repeats a small amount of text between adjacent chunks. Overlap can help preserve information that crosses a chunk boundary. Too much overlap creates duplicate content and increases storage.

Metadata

Metadata is information stored alongside a text chunk. Useful fields include the source filename, page number, section heading, document identifier, and chunk number. Metadata supports filtering, source citations, debugging, and traceability.

For example, a chunk might contain the sentence “FAISS supports efficient similarity search” and metadata such as source=`ai_engineering_guide.pdf`, page=7, section=`Vector Search`, chunk_id=42.

A robust ingestion pipeline should record which document version produced each chunk. When a PDF is replaced, the application should update or rebuild the associated vectors to avoid retrieving outdated passages.

7. Vector Search and FAISS

FAISS (Facebook AI Similarity Search) is a library for efficient similarity search and clustering of dense vectors. It is commonly used to build local vector-search systems and prototypes.

A basic FAISS workflow is: generate embeddings for document chunks, add the vectors to an index, embed a user query with the same model, search the index for nearest vectors, and map the returned vector IDs back to the original text chunks.

FAISS is a search library, not a complete document database. An application commonly stores chunk text and metadata separately, using IDs to connect them to vectors in the FAISS index.

Top-k retrieval

The parameter **k** controls how many nearest results are returned. If k=3, the retriever returns the three closest indexed vectors. A very small k may omit useful evidence; a very large k may add irrelevant passages. The best value depends on document structure, chunk size, and the task.

Exact and approximate search

Exact nearest-neighbour search compares a query against all indexed vectors and can be practical for smaller collections. Approximate indexes trade some search accuracy for speed and lower resource use on larger collections. Index choice should be guided by dataset size, latency requirements, memory, and acceptable recall.

8. Building a Reliable RAG Application

A practical pipeline

- **1. Load:** accept a PDF and extract text page by page.
- **2. Clean:** remove repeated page artifacts where safe, normalize whitespace, and preserve meaningful headings.
- **3. Split:** create chunks with a chosen size and overlap.
- **4. Embed:** generate vectors using one consistent embedding model.
- **5. Index:** store vectors and maintain a mapping to text and metadata.
- **6. Retrieve:** embed the question and search for the top-k relevant chunks.
- **7. Generate:** provide the question and retrieved passages to the LLM.
- **8. Cite:** show source filenames and page numbers when available.
- **9. Evaluate:** test retrieval and answer quality with a set of known questions.

Good answer behaviour

A document-grounded chatbot should distinguish evidence from general knowledge. If the retrieved passages do not answer the question, it should say that the uploaded documents do not provide enough information rather than inventing a document-based answer.

Retrieved document text must be treated as data, not as instructions. A PDF could contain malicious or irrelevant text that says to ignore the application's rules. The system prompt should tell the LLM to use retrieved text only as evidence for answering the user's question.

Evaluation questions

Evaluate a RAG system with questions whose answers are known from the source PDFs. Check whether the correct passage is retrieved, whether the final answer is supported by that passage, whether citations point to the right page, and whether the system admits when evidence is missing.

Useful metrics include retrieval recall at k, precision at k, answer faithfulness, answer relevance, latency, and cost. Human review remains valuable, especially for technical or high-impact use cases.

9. Glossary

AI: Artificial Intelligence; the broad field of systems that perform tasks associated with intelligence.

Chunk: A smaller text segment created from a longer document.

Context window: The maximum token context a model can process in a request.

Embedding: A numerical vector representation of text or another item.

FAISS: A library for efficient similarity search over dense vectors.

Hallucination: A model-generated claim that is unsupported or incorrect.

LLM: Large Language Model; a neural network trained to process and generate language.

Metadata: Information describing a document or chunk, such as source, page, or section.

OCR: Optical Character Recognition; converting text in images into machine-readable text.

RAG: Retrieval-Augmented Generation; retrieving external information before generating an answer.

Retriever: A component that searches a knowledge collection for relevant items.

Semantic similarity: Similarity in meaning rather than only exact word overlap.

Token: A unit of text processed by a language model.

Vector index: A data structure that supports searching vectors by similarity or distance.

10. Questions for testing your chatbot

1. What is the difference between AI, machine learning, and deep learning?

2. What is the role of self-attention in a Transformer?

3. Why does a RAG application split a PDF into chunks?

4. What is an embedding, and why should the same embedding model be used for documents and queries?

5. How does cosine similarity help retrieve relevant passages?

6. What does the top-k parameter control in vector search?

7. Why is metadata such as page number useful in a RAG application?

8. What is the difference between the ingestion phase and the question-answering phase?

9. Name two reasons a RAG system might return an incorrect answer.

10. What should a document-grounded chatbot do when the retrieved text does not contain the answer?

11. What is the difference between exact and approximate nearest-neighbour search?

12. Why might a scanned PDF require OCR before text extraction?

End of practice guide. Suggested exercise: ask your RAG chatbot each question above, then verify its answer against the relevant section of this PDF.